Project - High Level Design
on

Sports Planning Assistant Agent

Course Name: Agentic-AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ABHAY SINGH THAKUR	EN22CS304003
2	SARTHAK GOKHALE	EN22CS304057
3	HIMANSHI PORWAL	EN22CS306021
4	AKASH MUVEL	EN23EE3L1001
5	PRATHAM JOSHI	EN22CS306037

Group Name: ***Group 04D11***

*Project Number:****AAI-30***

Industry Mentor Name: ***Mr. Ashwin***

*University Mentor Name:****Prof. Nishant sir***

*Academic Year:****2022-2026***

1. Introduction

The **Sports Planning Assistant Agent** is an intelligent AI-based system designed to assist users in planning and organizing sports-related activities efficiently. The system utilizes an **agent-based architecture** combined with **Large Language Model (LLM) APIs** to automate complex planning tasks such as scheduling, task decomposition, and execution management.

Unlike traditional planning systems that require manual coordination, this system enables automated decision-making by converting high-level user goals into structured and actionable plans. The use of intelligent agents allows the system to dynamically analyze tasks, manage dependencies, and generate optimized execution sequences.

1.1 Scope of the Document

This document provides a detailed description of the system's architecture, design, and operational flow. It includes:

- Overall system design and structure
- Application workflow and processing steps
- Component-level design
- Data handling mechanisms
- API integrations
- Performance and security considerations

The document serves as a technical reference for understanding, evaluating, and extending the system.

1.2 Intended Audience

This document is intended for:

- Software developers and engineers
- AI/ML practitioners
- Academic evaluators and project reviewers
- Students studying intelligent systems

It provides both conceptual and technical insights into the system.

1.3 System Overview

The Sports Planning Assistant Agent is designed to process user queries related to sports planning and transform them into structured execution plans.

The system accepts natural language input such as *"Plan a football tournament"* and performs the following:

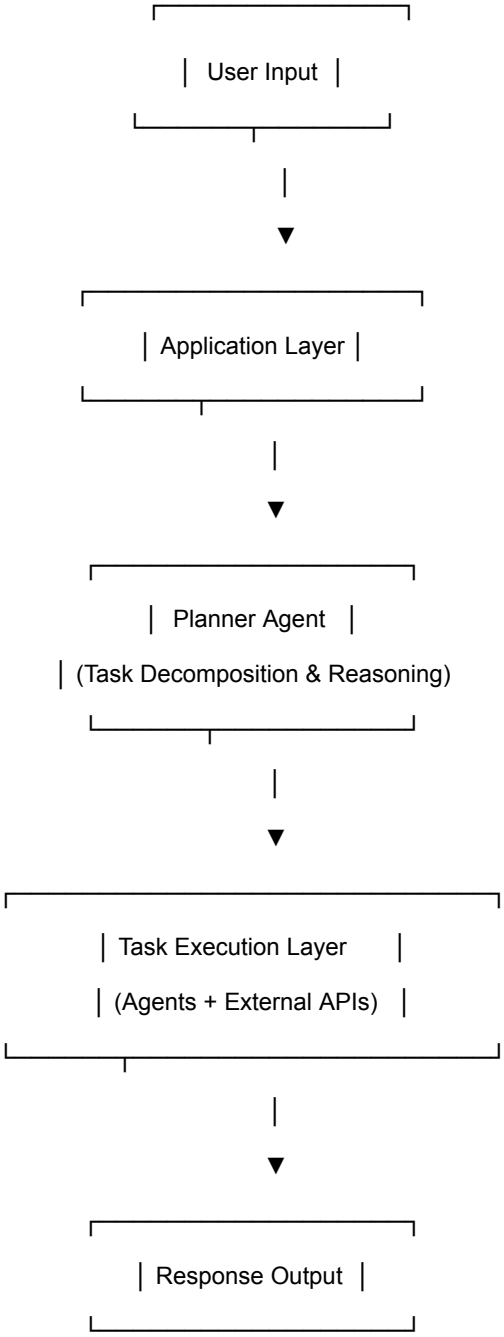
- Understands the user's intent
- Breaks the task into smaller sub-tasks
- Identifies dependencies among tasks
- Schedules tasks in logical order
- Executes tasks using agents and APIs
- Generates a structured and meaningful output

The core of the system is the **Planner Agent**, which enables intelligent decision-making and task orchestration.

2. System Design

2.1 Application Design

The system follows a **modular and agent-based architecture**, ensuring scalability, flexibility, and maintainability. Each component performs a specific function, allowing independent development and enhancement.



Architecture Explanation

- **User Input:** Provides high-level goal
- **Application Layer:** Manages system flow and routing
- **Planner Agent:** Performs task decomposition and reasoning
- **Task Execution Layer:** Executes tasks using agents and APIs
- **Response Output:** Displays final result

2.2 Process Flow

The system operates through the following steps:

1. The user submits a query in natural language
2. The application processes and interprets the input
3. The Planner Agent analyzes the request
4. The task is decomposed into smaller sub-tasks
5. Dependencies between tasks are identified
6. Tasks are scheduled in a logical order
7. Execution is performed using agents and APIs
8. Results are aggregated and formatted
9. Final output is presented to the user

This structured workflow ensures efficient and accurate planning.

2.3 Information Flow

The information flow in the system follows a sequential pattern:

- Input is received from the user
- Data is processed by the application layer
- Planner Agent generates structured tasks
- Tasks are executed using external tools
- Results are returned and formatted

The system operates in a **stateless manner**, meaning no data is stored between sessions.

2.4 Components Design

The system consists of the following key components:

1. User Interface

Provides a medium for user interaction. It accepts input and displays output.

2. Application Layer

Acts as the central controller, managing communication between all components.

3. Planner Agent

The core component responsible for:
Understanding user intent
Breaking tasks into sub-tasks
Managing dependencies
Generating execution plans

4. Task Execution Agents

a) Scheduling Agent

Handles time-related planning and sequencing of tasks.

b) Information Retrieval Agent

Fetches required data from external sources.

c) Recommendation Agent

Provides suggestions to optimize planning.

5. API Integration Layer

Facilitates communication with external APIs such as LLM and data services.

6. LLM Processing Unit

Provides natural language understanding and reasoning capabilities.

7. Configuration Module

Manages environment variables and API keys securely.

2.5 Key Design Considerations

The system design is guided by the following principles:

- **Scalability:** Ability to handle increasing complexity
- **Performance:** Fast response time through efficient processing
- **Security:** Protection of API keys and sensitive data
- **Modularity:** Independent components for easy maintenance
- **Extensibility:** Capability to add new features and agents
- **Reliability:** Consistent and accurate output generation

2.6 API Catalogue

API Name	Purpose	Description
LLM API	Natural language processing	Generates responses and reasoning
Search API	Data retrieval	Fetches real-time information
Sports API	Match data	Provides sports-related data
Weather API	Weather conditions	Assists in planning outdoor events

3. Data Design

The data design of the Sports Planning Assistant Agent is structured to support real-time processing, flexibility, and simplicity. Unlike traditional systems that rely heavily on databases, this system processes data dynamically without persistent storage. The primary focus is on handling user input efficiently and transforming it into actionable steps using intelligent agents.

The system receives input in natural language form, which is inherently unstructured. This input is then processed by the application layer and converted into a structured format that can be understood by the Planner Agent. The planner agent further decomposes this structured data into smaller tasks, which are executed sequentially or in parallel depending on dependencies.

.User Input (Natural Language)

↓

Application Layer Processing

↓

Planner Agent (Task Breakdown)

↓

Structured Task List (Temporary Data)

↓

Execution Agents + APIs

↓

Response Generation

↓

Output to User

3.1 Data Model

The data model used in the system is a hybrid combination of unstructured and semi-structured data. User input is received as unstructured text, which is then processed and converted into semi-structured formats such as lists, dictionaries, or JSON-like objects.

For example, when a user provides a query such as "Plan a cricket match," the system interprets this input and converts it into a structured format containing multiple steps. These steps may include selecting a venue, scheduling a time, organizing teams, and preparing match logistics.

Example:

```
{
 "task": "Plan Match",
 "steps": [
   "Select venue",
   "Schedule time",
   "Organize teams"
 ]
}
```

This model allows flexibility while maintaining clarity during task execution.

3.2 Data Access Mechanism

The data access mechanism in the system is designed to be simple and efficient. Since there is no database, all data is accessed directly through user input and external APIs. The system uses in-memory processing to handle all data operations.

When a user enters a query, the system processes it immediately without storing it. The application layer forwards the input to the planner agent, which generates a task list. If additional information is required, the system interacts with external APIs such as LLM APIs or search services.

This ensures:

- Faster performance
- Reduced complexity
- Real-time execution

3.3 Data Retention Policies

The system follows a strict no-retention policy for data. This means that no user data is stored permanently, and all information is processed only during execution. Once the response is generated and delivered to the user, all intermediate data is discarded.

This approach provides several advantages. First, it ensures user privacy, as no personal or sensitive data is stored. Second, it reduces storage requirements, making the system lightweight and efficient. Third, it eliminates the need for data management tasks such as backup, recovery, and migration.

This ensures:

- High privacy
- No data leakage risk
- Minimal storage overhead

3.4 Data Migration

Data migration is not applicable in the current system, as there is no database or persistent storage. Since all data is processed in real-time and discarded afterward, there is no need to transfer or migrate data between systems.

In future versions, if a database is introduced, data migration strategies may be required to handle the transition from a stateless system to a stateful one.

4. Interfaces

The Sports Planning Assistant Agent provides a simple yet effective interface for user interaction. The primary interface used in the current implementation is a **Command-Line Interface (CLI)**, which allows users to input queries and receive responses directly through the terminal. This design keeps the system lightweight and easy to use while focusing on core functionality.

The CLI interface acts as the entry point of the system, where the user provides a high-level goal in natural language. The system then processes this input using the application layer and planner agent to generate a structured output. The simplicity of the CLI ensures that users can interact with the system without requiring any specialized tools or graphical environments.

One of the key advantages of using a CLI is its efficiency. It requires minimal system resources and allows quick testing and execution of the application. It is particularly useful during development and debugging phases, as developers can easily input different queries and observe system behavior in real time.

User

▼

Command Line Interface

▼

Application Layer

▼

Planner Agent

▼

Execution Agents & APIs

▼

Response Output (CLI Display)

Types of Interfaces

1. Command-Line Interface (Current)

- Simple and lightweight
- Easy to implement and use
- Suitable for development and testing

2. Web Interface (Future Scope)

- Graphical user interface
- More user-friendly
- Supports forms and dashboards

3. Mobile Interface (Future Scope)

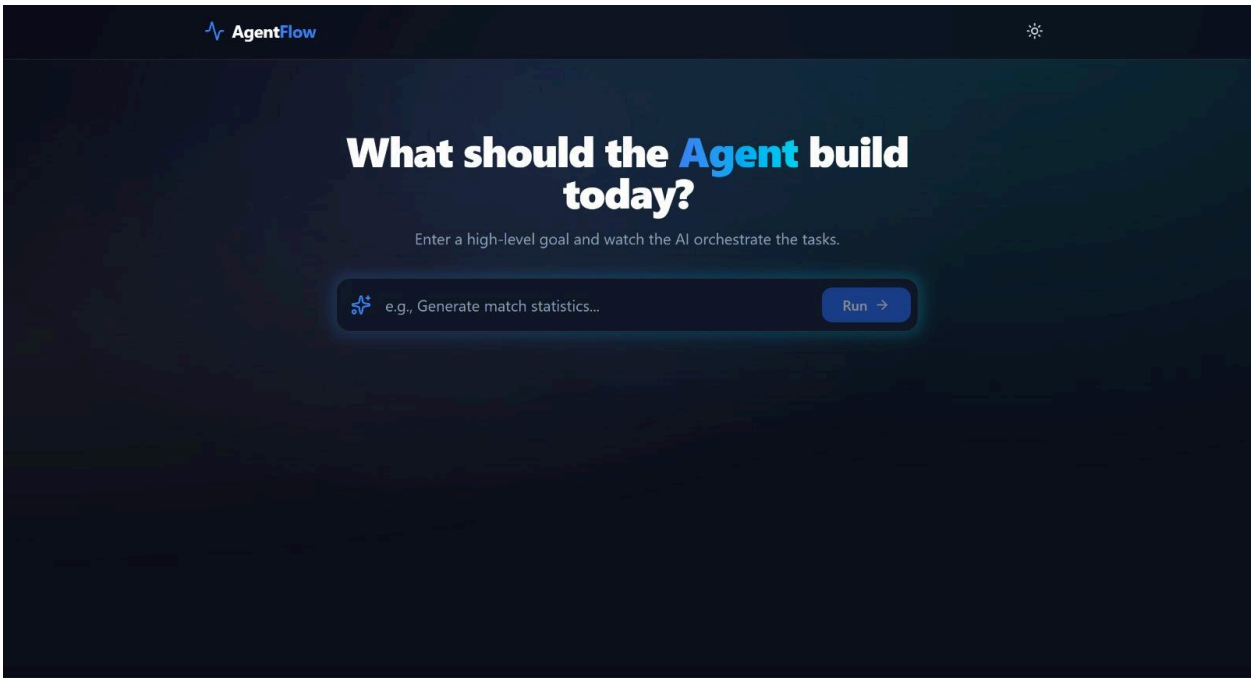
- Accessible on smartphones
- Useful for real-time planning
- Enhances portability

5. State and Session Management

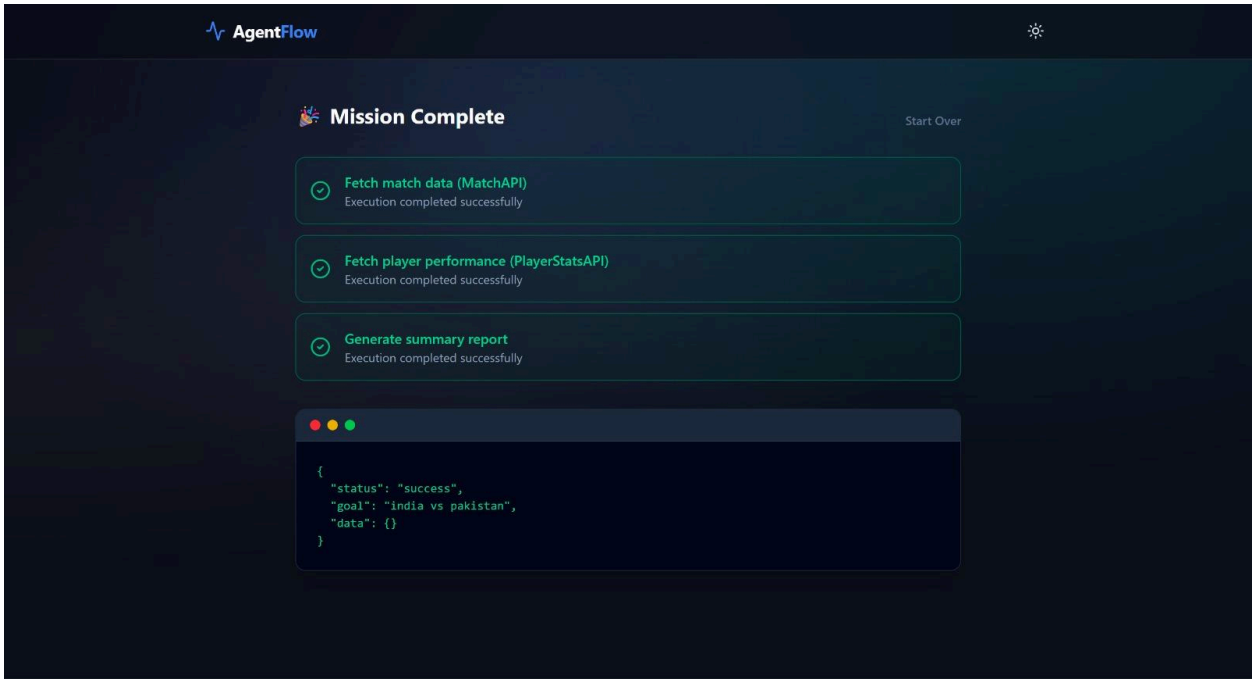
The system follows a **stateless architecture**.

The Sports Planning Assistant Agent is designed as a **stateless system**, meaning it does not maintain any session information or memory of previous interactions. Each user request is processed independently, and the system generates a response based solely on the current input without relying on past data.

In this approach, when a user submits a query through the interface, the system processes the input, generates a structured plan, and returns the output. Once the response is delivered, all intermediate data is discarded. The system does not store any information about the user or previous requests, ensuring that each interaction is isolated.



- Each request is processed independently
- No session history is maintained
- No user data is stored between requests



Advantages:

- Simpler design
- Better scalability
- No session management overhead

Future Improvements

- Add session-based memory
- Use vector databases for context
- Enable conversational continuity

6. Caching

Currently, caching is not implemented in the system.

Caching is a technique used to improve system performance by storing frequently accessed data so that it can be reused without repeated processing. In the current implementation of the Sports Planning Assistant Agent, **caching is not implemented**, and every user request is processed independently in real-time.

The system relies on direct interaction with the Planner Agent and LLM APIs to generate responses. Each time a user submits a query, the system performs task decomposition, reasoning, and execution from scratch. While this ensures accuracy and freshness of results, it may lead to repeated computation for similar queries.

Caching can be implemented at different levels:

- **Response-Level Caching:**
  Stores complete outputs for repeated queries.
- **API Response Caching:**
  Stores results fetched from external APIs.
- **Task-Level Caching:**
  Stores intermediate results such as task decomposition.

User Request
|
▼
Check Cache (Not Implemented)
|
▼
Planner Agent + LLM Processing
|
▼
Generate Response
|
▼
(Optional Future: Store in Cache)

7. Non-Functional Requirements

The system is designed to meet important non-functional requirements:

Non-functional requirements define the quality attributes and operational standards of the system. For the Sports Planning Assistant Agent, these requirements ensure that the system performs efficiently, remains reliable, and provides a good user experience. Unlike functional requirements, which describe what the system does, non-functional requirements focus on how well the system performs its tasks.

The system is designed with a strong emphasis on **performance, scalability, reliability, and usability**. Since the application processes user queries in real-time using LLM APIs, it must generate responses quickly and efficiently. The lightweight architecture and absence of a database help reduce latency and improve response time.

Requirement	Description
Performance	Fast response, real-time processing
Scalability	Easy to expand with new agents
Reliability	Consistent and accurate outputs
Usability	Simple CLI interface
Maintainability	Modular and easy to update

1. Performance

The system is optimized for quick response generation using real-time processing and minimal overhead.

2. Scalability

The architecture allows easy addition of new components such as agents and APIs without major changes.

3. Reliability

The system ensures consistent and stable output generation for different types of user queries.

4. Usability

The CLI interface makes the system simple and easy to use, even for beginners.

5. Maintainability

The modular design ensures that updates and modifications can be done easily.

8. Security Aspects

Security is an important part of the Sports Planning Assistant Agent as it involves interaction with external APIs and handling of sensitive information. The system is designed to ensure safe execution while maintaining user privacy. Since it does not store any user data and processes everything in real-time, the risk of data leakage is minimized.

Key Security Measures

- **Environment Variables**
  - API keys are stored in .env files instead of code
  - Prevents exposure of sensitive information
- **Stateless Design**
  - No session or user data is stored
  - Each request is processed independently
- **No Data Retention**
  - User inputs and outputs are not saved
  - Ensures privacy and reduces security risks
- **Secure API Communication**
  - Uses authenticated API calls
  - Interacts only with trusted services
- **Input Handling**
  - Ensures safe processing of user input
  - Avoids unexpected system behavior

9. Performance Aspects

Performance is a key factor in the Sports Planning Assistant Agent, as the system is expected to generate responses quickly and efficiently. The system is designed with a lightweight architecture that processes user queries in real-time without relying on any database. This reduces delays and allows faster execution of tasks. However, since the system depends on external APIs such as LLM services, performance may vary based on API response time.

Key Performance Features

- **Real-Time Processing**
  - Processes user queries instantly
  - No delay due to data storage or retrieval
- **Lightweight Architecture**
  - No database integration
  - Minimal system overhead
- **Efficient Task Execution**
  - Planner Agent breaks tasks into steps
  - Tasks are executed in a logical sequence
- **Direct API Communication**
  - Faster interaction with LLM APIs
  - Reduces unnecessary processing layers

10. References

The development of the Sports Planning Assistant Agent is based on various technologies, frameworks, and concepts related to Artificial Intelligence and agent-based systems. The following references were used as guidance for design and implementation:

Technical References

- Large Language Model (LLM) concepts and documentation
- OpenAI / similar LLM API documentation
- Python programming documentation

Framework References

- CrewAI framework for agent-based systems
- LangChain framework for LLM integration

Conceptual References

- Agent-based system design principles
- Task decomposition and planning in AI
- Multi-agent system architecture

Online Resources

- Official documentation of APIs and libraries used
- Tutorials and guides related to AI agents and planning systems
- Research articles on intelligent automation